

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 from sklearn.ensemble import RandomForestRegressor
6 from sklearn.preprocessing import MinMaxScaler
7 from scipy.optimize import minimize
8 import warnings
9 warnings.filterwarnings('ignore')
10
11 # Install yfinance if needed
12 try:
13     import yfinance as yf
14 except ImportError:
15     import subprocess
16     import sys
17     subprocess.check_call([sys.executable, "-m", "pip", "install", "yfinance"])
18     import yfinance as yf
19
20 # =====
21 # 1. TRADITIONAL MEAN-VARIANCE OPTIMIZATION (MARKOWITZ)
22 # =====
23
24 def mean_variance_opt(data):
25     returns = data.pct_change().dropna()
26     mu = returns.mean().values
27     cov = returns.cov().values
28     n = len(mu)
29
30     def objective(weights):
31         port_return = np.dot(weights, mu)
32         port_vol = np.sqrt(np.dot(weights.T, np.dot(cov, weights)))
33         return -port_return / port_vol
34
35     cons = ({'type': 'eq', 'fun': lambda x: np.sum(x) - 1},)
36     bounds = tuple((0, 1) for _ in range(n))
37     x0 = np.ones(n) / n
38
39     result = minimize(objective, x0, method='SLSQP', bounds=bounds, constraints=cons)
40     optimal_weights = result.x
41     port_return = np.dot(optimal_weights, mu)
42     port_vol = np.sqrt(np.dot(optimal_weights.T, np.dot(cov, optimal_weights)))
43     sharpe = port_return / port_vol
44
45     return optimal_weights, port_return, port_vol, sharpe
46
47 # =====
48 # 2. MACHINE LEARNING OPTIMIZATION (RANDOM FOREST)
49 # =====
50
51 def create_features(data, lookback=10):
52     features = []
53     returns = data.pct_change().dropna()

```

```

54
55     for i in range(lookback, len(data)):
56         price_features = []
57         for col in data.columns:
58             recent_prices = data[col].iloc[i-lookback:i].values
59             normalized_prices = recent_prices / recent_prices[0]
60             price_features.extend(normalized_prices)
61
62         return_features = []
63         for col in returns.columns:
64             recent_returns = returns[col].iloc[i-lookback:i].values
65             return_features.extend(recent_returns)
66
67         sma_5 = data.iloc[i-5:i].mean().values
68         volatility = returns.iloc[i-10:i].std().values
69
70         combined_features = np.concatenate([price_features, return_features, sma_5, volatility])
71         features.append(combined_features)
72
73     return np.array(features)
74
75 def ml_portfolio_optimization(data, lookback=10):
76     X = create_features(data, lookback)
77     returns = data.pct_change().dropna()
78     y = returns.iloc[lookback:].values
79     models = []
80
81     for asset_idx in range(data.shape[1]):
82         model = RandomForestRegressor(n_estimators=100, random_state=42)
83         split_idx = int(0.8 * len(X))
84         X_train = X[:split_idx]
85         y_train = y[:split_idx, asset_idx]
86         model.fit(X_train, y_train)
87         models.append(model)
88
89     expected_returns = np.array([model.predict(X[-1].reshape(1, -1))[0] for model in models])
90     recent_volatility = returns.tail(30).std().values
91     risk_adjusted_returns = expected_returns / (recent_volatility + 1e-8)
92     weights = np.maximum(risk_adjusted_returns, 0)
93     weights = weights / np.sum(weights) if np.sum(weights) > 0 else np.ones(len(weights)) / len(weights)
94
95     portfolio_return = np.dot(weights, expected_returns)
96     portfolio_volatility = np.sqrt(np.dot(weights.T, np.dot(returns.cov().values, weights)))
97     sharpe_ratio = portfolio_return / portfolio_volatility if portfolio_volatility > 0 else 0
98
99     return weights, portfolio_return, portfolio_volatility, sharpe_ratio
100
101 # =====
102 # 3. DEEP LEARNING OPTIMIZATION (NEURAL NETWORK)
103 # =====
104
105 def simple_neural_network_portfolio(data, hidden_neurons=32, learning_rate=0.001, epochs=50):
106     returns = data.pct_change().dropna()
107     features = []
108     for i in range(10, len(data)):

```

```

109     ma_5 = data.iloc[i-5:i].mean().values
110     ma_10 = data.iloc[i-10:i].mean().values
111     vol = returns.iloc[i-10:i].std().values
112     momentum = (data.iloc[i] / data.iloc[i-5] - 1).values
113     price_ratios = (data.iloc[i] / data.iloc[i-1]).values
114     feature_vector = np.concatenate([ma_5, ma_10, vol, momentum, price_ratios])
115     features.append(feature_vector)
116
117 X = np.array(features)
118 y = returns.iloc[10:].values
119 split_idx = int(0.8 * len(X))
120 X_train, X_test = X[:split_idx], X[split_idx:]
121 y_train, y_test = y[:split_idx], y[split_idx:]
122
123 scaler = MinMaxScaler()
124 X_train_scaled = scaler.fit_transform(X_train)
125 X_test_scaled = scaler.transform(X_test)
126
127 input_size = X_train_scaled.shape[1]
128 output_size = y_train.shape[1]
129
130 np.random.seed(42)
131 W1 = np.random.randn(input_size, hidden_neurons) * 0.1
132 b1 = np.zeros((1, hidden_neurons))
133 W2 = np.random.randn(hidden_neurons, output_size) * 0.1
134 b2 = np.zeros((1, output_size))
135
136 def relu(x):
137     return np.maximum(0, x)
138
139 for epoch in range(epochs):
140     z1 = np.dot(X_train_scaled, W1) + b1
141     a1 = relu(z1)
142     z2 = np.dot(a1, W2) + b2
143     predictions = z2
144     loss = np.mean((predictions - y_train) ** 2)
145     if epoch % 20 == 0:
146         print(f" Epoch {epoch}, Loss: {loss:.6f}")
147
148 last_features = X_test_scaled[-1:, :]
149 z1 = np.dot(last_features, W1) + b1
150 a1 = relu(z1)
151 z2 = np.dot(a1, W2) + b2
152 predicted_returns = z2[0]
153
154 exp_returns = np.exp(predicted_returns - np.max(predicted_returns))
155 weights = exp_returns / np.sum(exp_returns)
156 weights = np.abs(weights) / np.sum(np.abs(weights))
157
158 portfolio_return = np.dot(weights, predicted_returns)
159 portfolio_volatility = np.sqrt(np.dot(weights.T, np.dot(returns.cov().values, weights)))
160 sharpe_ratio = portfolio_return / portfolio_volatility if portfolio_volatility > 0 else 0
161
162 return weights, portfolio_return, portfolio_volatility, sharpe_ratio
163

```

```

164 # =====
165 # 4. REINFORCEMENT LEARNING (PPO) - FIXED
166 # =====
167
168 class PPOPortfolioAgent:
169     def __init__(self, n_assets, lr_actor=0.001, lr_critic=0.001, gamma=0.95, epsilon_clip=0.1,
170                 self.n_assets = n_assets
171                 self.lr_actor = lr_actor
172                 self.lr_critic = lr_critic
173                 self.gamma = gamma
174                 self.epsilon_clip = epsilon_clip
175                 self.epochs = epochs
176                 self.actor_weights = np.random.randn(n_assets * 3, n_assets) * 0.001
177                 self.actor_bias = np.zeros(n_assets)
178                 self.critic_weights = np.random.randn(n_assets * 3, 1) * 0.001
179                 self.critic_bias = 0.0
180                 self.states = []
181                 self.actions = []
182                 self.rewards = []
183                 self.log_probs = []
184                 self.values = []
185
186     def get_action(self, state):
187         if np.any(np.isnan(state)) or np.any(np.isinf(state)):
188             state = np.nan_to_num(state, nan=0.0, posinf=1.0, neginf=-1.0)
189         logits = np.dot(state, self.actor_weights) + self.actor_bias
190         logits = np.clip(logits, -10, 10)
191         logits = logits - np.max(logits)
192         exp_logits = np.exp(logits)
193         probs = exp_logits / (np.sum(exp_logits) + 1e-8)
194         if np.any(np.isnan(probs)) or np.sum(probs) == 0:
195             probs = np.ones(self.n_assets) / self.n_assets
196         else:
197             probs = probs / np.sum(probs)
198         probs = 0.95 * probs + 0.05 * np.ones(self.n_assets) / self.n_assets
199         log_prob = np.sum(np.log(np.clip(probs, 1e-10, 1.0)))
200         return probs, log_prob
201
202     def get_value(self, state):
203         if np.any(np.isnan(state)) or np.any(np.isinf(state)):
204             state = np.nan_to_num(state, nan=0.0, posinf=1.0, neginf=-1.0)
205         value = np.dot(state, self.critic_weights).flatten()[0] + self.critic_bias
206         value = np.clip(value, -10, 10)
207         return value
208
209     def store_transition(self, state, action, reward, log_prob):
210         if np.isnan(reward) or np.isinf(reward): reward = 0.0
211         if np.isnan(log_prob) or np.isinf(log_prob): log_prob = -1.0
212         self.states.append(state)
213         self.actions.append(action)
214         self.rewards.append(reward)
215         self.log_probs.append(log_prob)
216         self.values.append(self.get_value(state))
217
218     def compute_returns(self):
219         if len(self.rewards) == 0: return np.array([1])

```

```

219         if len(self.rewards) == 0: return np.array([])
220     returns = []
221     G = 0
222     for reward in reversed(self.rewards):
223         G = reward + self.gamma * G
224         returns.insert(0, G)
225     returns = np.array(returns)
226     if len(returns) > 1 and np.std(returns) > 1e-8:
227         returns = (returns - np.mean(returns)) / (np.std(returns) + 1e-8)
228     return returns
229
230     def update(self):
231         if len(self.states) < 2: return
232         returns = self.compute_returns()
233         if len(returns) == 0: return
234         states = np.array(self.states)
235         values = np.array(self.values)
236         advantages = returns - values
237         for i, state in enumerate(states):
238             if np.any(np.isnan(state)) or np.any(np.isinf(state)): continue
239             advantage = advantages[i]
240             if not np.isnan(advantage) and not np.isinf(advantage):
241                 gradient = np.clip(advantage * state.reshape(-1, 1), -0.1, 0.1)
242                 self.actor_weights += self.lr_actor * gradient
243                 value_error = returns[i] - values[i]
244                 if not np.isnan(value_error) and not np.isinf(value_error):
245                     critic_gradient = np.clip(value_error * state.reshape(-1, 1), -0.1, 0.1)
246                     self.critic_weights += self.lr_critic * critic_gradient
247         self.states = []; self.actions = []; self.rewards = []; self.log_probs = []; self.values = []
248
249     class PortfolioEnvironment:
250     def __init__(self, price_data, initial_capital=10000, commission=0.0005):
251         self.price_data = price_data.copy()
252         self.initial_capital = initial_capital
253         self.commission = commission
254         self.n_assets = price_data.shape[1]
255         self.n_steps = len(price_data)
256         self.reset()
257
258     def reset(self):
259         self.current_step = 0
260         self.portfolio_value = self.initial_capital
261         self.holdings = np.ones(self.n_assets) / self.n_assets
262         return self._get_state()
263
264     def _get_state(self):
265         if self.current_step >= self.n_steps:
266             return np.zeros(self.n_assets * 3)
267         prices = self.price_data.iloc[self.current_step].values
268         normalized_prices = prices / (prices[0] + 1e-8)
269         if self.current_step > 0:
270             prev_prices = self.price_data.iloc[self.current_step - 1].values
271             returns = (prices - prev_prices) / (prev_prices + 1e-8)
272             returns = np.clip(returns, -0.2, 0.2)
273         else:
274             returns = np.zeros(self.n_assets)

```

```

275     portfolio_weights = self.holdings / (np.sum(self.holdings) + 1e-8)
276     state = np.concatenate([normalized_prices, returns, portfolio_weights])
277     return np.nan_to_num(state, nan=0.0, posinf=1.0, neginf=-1.0)
278
279 def step(self, weights):
280     if self.current_step >= self.n_steps - 1:
281         return self._get_state(), 0.0, True
282     if np.any(np.isnan(weights)) or np.sum(weights) == 0:
283         weights = np.ones(self.n_assets) / self.n_assets
284     else:
285         weights = weights / np.sum(weights)
286     current_prices = self.price_data.iloc[self.current_step].values
287     next_prices = self.price_data.iloc[self.current_step + 1].values
288     price_changes = (next_prices - current_prices) / (current_prices + 1e-8)
289     price_changes = np.clip(price_changes, -0.5, 0.5)
290     portfolio_return = np.dot(weights, price_changes)
291     weight_changes = np.abs(weights - self.holdings)
292     transaction_cost = np.sum(weight_changes) * self.commission
293     reward = portfolio_return - transaction_cost
294     reward = np.clip(reward, -0.1, 0.1)
295     self.holdings = weights.copy()
296     self.current_step += 1
297     done = self.current_step >= self.n_steps - 1
298     return self._get_state(), reward, done
299
300 def train_ppo_portfolio(price_data, episodes=20, max_steps=50):
301     env = PortfolioEnvironment(price_data)
302     agent = PPOPortfolioAgent(n_assets=price_data.shape[1])
303     episode_rewards = []
304     for episode in range(episodes):
305         state = env.reset()
306         episode_reward = 0
307         step_count = 0
308         done = False
309         while not done and step_count < max_steps:
310             action, log_prob = agent.get_action(state)
311             next_state, reward, done = env.step(action)
312             agent.store_transition(state, action, reward, log_prob)
313             episode_reward += reward
314             state = next_state
315             step_count += 1
316         agent.update()
317         episode_rewards.append(episode_reward)
318         if (episode + 1) % 5 == 0:
319             recent_rewards = episode_rewards[-5:]
320             avg_reward = np.mean(recent_rewards) if recent_rewards else 0.0
321             print(f" Episode {episode+1}/{episodes} | Avg Reward: {avg_reward:.4f}")
322     return agent, episode_rewards
323
324 # =====
325 # 5. NEW: HYBRID ENSEMBLE WITH DYNAMIC SWITCHING
326 # =====
327
328 def hybrid_ensemble_optimization(data):
329     print("\n[NEW] Running Hybrid Ensemble Model with Dynamic Switching...")

```

```

330 returns = data.pct_change().dropna()
331 current_volatility = returns.std().mean()
332 current_trend = (data.iloc[-1] / data.iloc[-5] - 1).mean()
333
334 scores = {
335     'Markowitz': 1.0 / (1.0 + current_volatility),
336     'ML': 1.0 / (1.0 + abs(current_trend)),
337     'DL': 1.0 / (1.0 + current_volatility * abs(current_trend)),
338     'RL': 0.8 + 0.2 * np.random.rand()
339 }
340
341 total_score = sum(scores.values())
342 switching_probs = {k: v / total_score for k, v in scores.items()} if total_score > 0 else
343
344 trad_w, _, _, _ = mean_variance_opt(data)
345 ml_w, _, _, _ = ml_portfolio_optimization(data)
346 dl_w, _, _, _ = simple_neural_network_portfolio(data)
347 agent, _ = train_ppo_portfolio(data, episodes=10)
348 final_state = np.concatenate([
349     data.iloc[-1].values / data.iloc[-1].values.mean(),
350     np.zeros(len(data.columns)),
351     np.ones(len(data.columns)) / len(data.columns)
352 ])
353 rl_w, _ = agent.get_action(final_state)
354
355 ensemble_weights = (
356     switching_probs['Markowitz'] * trad_w +
357     switching_probs['ML'] * ml_w +
358     switching_probs['DL'] * dl_w +
359     switching_probs['RL'] * rl_w
360 )
361 ensemble_weights /= np.sum(ensemble_weights)
362
363 portfolio_return = np.dot(ensemble_weights, returns.mean().values)
364 portfolio_volatility = np.sqrt(np.dot(ensemble_weights.T, np.dot(returns.cov().values, ensemble_weights)))
365 sharpe_ratio = portfolio_return / portfolio_volatility if portfolio_volatility > 0 else 0
366
367 return ensemble_weights, portfolio_return, portfolio_volatility, sharpe_ratio, switching_probs
368
369 # =====
370 # 6. NEW: MULTI-OBJECTIVE ESG OPTIMIZATION
371 # =====
372
373 def multi_objective_esg_optimization(data, esg_scores=None, alpha=0.5):
374     print("\n[NEW] Running Multi-Objective Optimization with ESG Factors...")
375     if esg_scores is None:
376         esg_scores = {ticker: np.random.uniform(0, 100) for ticker in data.columns}
377         print("    Using mock ESG scores for demonstration.")
378
379     esg_values = np.array([esg_scores[ticker] for ticker in data.columns])
380     esg_values = (esg_values - esg_values.min()) / (esg_values.max() - esg_values.min() + 1e-8)
381
382     returns = data.pct_change().dropna()
383     mu = returns.mean().values
384     cov = returns.cov().values

```

```

385     n = len(mu)
386
387     def objective(weights):
388         port_return = np.dot(weights, mu)
389         port_vol = np.sqrt(np.dot(weights.T, np.dot(cov, weights)))
390         portfolio_esg = np.dot(weights, esg_values)
391         sharpe_component = port_return / port_vol if port_vol > 0 else 0
392         return -(alpha * portfolio_esg + (1 - alpha) * sharpe_component)
393
394     cons = ({'type': 'eq', 'fun': lambda x: np.sum(x) - 1},)
395     bounds = tuple((0, 1) for _ in range(n))
396     x0 = np.ones(n) / n
397
398     result = minimize(objective, x0, method='SLSQP', bounds=bounds, constraints=cons)
399     optimal_weights = result.x
400
401     portfolio_return = np.dot(optimal_weights, mu)
402     portfolio_vol = np.sqrt(np.dot(optimal_weights.T, np.dot(cov, optimal_weights)))
403     portfolio_esg = np.dot(optimal_weights, esg_values)
404     sharpe = portfolio_return / portfolio_vol if portfolio_vol > 0 else 0
405
406     return optimal_weights, portfolio_return, portfolio_vol, sharpe, portfolio_esg
407
408 # =====
409 # MAIN COMPARISON FUNCTION (FULLY CORRECTED & VISUALLY IMPROVED)
410 # =====
411
412 def compare_all_methods(data, esg_scores=None):
413     print("Portfolio Optimization Comparison - All Methods")
414     print("=" * 80)
415
416     print("\n1. Traditional Mean-Variance Optimization...")
417     trad_weights, trad_ret, trad_vol, trad_sharpe = mean_variance_opt(data)
418
419     print("2. Machine Learning Optimization (Random Forest)...")
420     ml_weights, ml_ret, ml_vol, ml_sharpe = ml_portfolio_optimization(data)
421
422     print("3. Deep Learning Optimization (Neural Network)...")
423     dl_weights, dl_ret, dl_vol, dl_sharpe = simple_neural_network_portfolio(data)
424
425     print("4. Reinforcement Learning Optimization (PPO)...")
426     agent, rewards = train_ppo_portfolio(data, episodes=30)
427     final_state = np.concatenate([
428         data.iloc[-1].values / data.iloc[-1].values.mean(),
429         np.zeros(len(data.columns)),
430         np.ones(len(data.columns)) / len(data.columns)
431     ])
432     rl_weights, _ = agent.get_action(final_state)
433     rl_returns_proxy = data.pct_change().dropna().mean().values
434     rl_ret = np.dot(rl_weights, rl_returns_proxy)
435     rl_vol = np.sqrt(np.dot(rl_weights.T, np.dot(data.pct_change().dropna().cov().values, rl_w
436     rl_sharpe = rl_ret / rl_vol if rl_vol > 0 else 0
437
438     print("5. Hybrid Ensemble Model with Dynamic Switching...")
439     ens_weights, ens_ret, ens_vol, ens_sharpe, switching_probs = hybrid_ensemble_optimization(

```

```

440
441 print("6. Multi-Objective Optimization with ESG Factors...")
442 esg_weights, esg_ret, esg_vol, esg_sharpe, portfolio_esg_score = multi_objective_esg_optim:
443
444 results = {
445     'Traditional': {'weights': trad_weights, 'return': trad_ret, 'vol': trad_vol, 'sharpe'
446     'Machine Learning': {'weights': ml_weights, 'return': ml_ret, 'vol': ml_vol, 'sharpe':
447     'Deep Learning': {'weights': dl_weights, 'return': dl_ret, 'vol': dl_vol, 'sharpe': dl_
448     'Reinforcement Learning': {'weights': rl_weights, 'return': rl_ret, 'vol': rl_vol, 'sh:
449     'Hybrid Ensemble': {'weights': ens_weights, 'return': ens_ret, 'vol': ens_vol, 'sharpe
450     'Multi-Obj ESG': {'weights': esg_weights, 'return': esg_ret, 'vol': esg_vol, 'sharpe':
451 }
452
453 # === SUMMARY TABLE ===
454 print("\n" + "=" * 80)
455 print("\n📊 PORTFOLIO OPTIMIZATION RESULTS COMPARISON")
456 print("=" * 80)
457 summary_data = []
458 methods_all = list(results.keys())
459 for method in methods_all:
460     r = results[method]
461     esg_val = r.get('esg_score', np.nan)
462     summary_data.append({
463         'Method': method,
464         'Return (%)': r['return'] * 100,
465         'Volatility (%)': r['vol'] * 100,
466         'Sharpe Ratio': r['sharpe'],
467         'ESG Score': esg_val
468     })
469 summary_df = pd.DataFrame(summary_data)
470 print("\n📈 Performance Summary:")
471 print(summary_df.round(4).to_string(index=False))
472
473 # === ALLOCATION TABLE ===
474 print("\n📊 Allocation Weights (%):")
475 alloc_data = []
476 for i, asset in enumerate(data.columns):
477     row = {'Asset': asset}
478     for method in methods_all:
479         w = results[method]['weights'][i] * 100
480         row[method] = f"{w:.2f}%"
481     alloc_data.append(row)
482 alloc_df = pd.DataFrame(alloc_data)
483 print(alloc_df.to_string(index=False))
484
485 # === ORIGINAL ALLOCATION BAR CHART ===
486 plt.figure(figsize=(12, 7))
487 assets = data.columns
488 x = np.arange(len(assets))
489 width = 0.13
490 colors = plt.cm.tab10.colors[:6]
491 for i, method in enumerate(methods_all):
492     w = results[method]['weights'] * 100
493     plt.bar(x + i*width, w, width, label=method, color=colors[i], alpha=0.85)
494 plt.xlabel('Assets', fontsize=12, fontweight='bold')

```

```

495 plt.ylabel('Allocation (%)', fontsize=12, fontweight='bold')
496 plt.title('Portfolio Allocation Comparison: All Six Methods', fontsize=14, fontweight='bold')
497 plt.xticks(x + width*2.5, assets, rotation=45, ha='right', fontsize=11)
498 plt.legend(loc='upper left', bbox_to_anchor=(1.05, 1), fontsize=9)
499 plt.grid(alpha=0.3, linestyle='--')
500 plt.tight_layout()
501 plt.show()
502
503 # === ENHANCED RISK VS RETURN SCATTER PLOT ===
504 plt.figure(figsize=(12, 8))
505 vol_vals = [results[m]['vol'] * 100 for m in methods_all]
506 ret_vals = [results[m]['return'] * 100 for m in methods_all]
507 sharpe_vals = [results[m]['sharpe'] for m in methods_all]
508
509 # Define performance tiers
510 def get_color(sr):
511     if sr >= 1.5:
512         return 'green'
513     elif sr >= 1.0:
514         return 'blue'
515     elif sr >= 0.5:
516         return 'orange'
517     else:
518         return 'red'
519
520 # Plot points with performance-based coloring
521 for i, method in enumerate(methods_all):
522     color = get_color(sharpe_vals[i])
523     marker = 'o' if sharpe_vals[i] >= 1.5 else 's' if sharpe_vals[i] >= 1.0 else '^' if sharpe_vals[i] >= 0.5 else 'x'
524     size = 200 if sharpe_vals[i] >= 1.5 else 150
525
526     plt.scatter(vol_vals[i], ret_vals[i],
527                 s=size,
528                 c=color,
529                 marker=marker,
530                 edgecolor='black',
531                 linewidth=1.5,
532                 alpha=0.8,
533                 label=f"{method[:8]} (SR={sharpe_vals[i]:.2f})")
534
535     # Add annotation
536     plt.annotate(method[:8],
537                 (vol_vals[i], ret_vals[i]),
538                 xytext=(5, 5),
539                 textcoords='offset points',
540                 fontsize=10,
541                 fontweight='bold',
542                 bbox=dict(boxstyle="round,pad=0.3", facecolor="white", alpha=0.8))
543
544 # Set reasonable limits (exclude extreme outliers)
545 vol_max = max(vol_vals) * 1.1 if max(vol_vals) > 0 else 1.0
546 ret_max = max(ret_vals) * 1.1 if max(ret_vals) > 0 else 1.0
547 plt.xlim(0, vol_max)
548 plt.ylim(0, ret_max)
549
550 # Add efficient frontier approximation

```

```

550 # Add efficient frontier approximation
551 if len(methods_all) > 1:
552     sorted_indices = np.argsort(vol_vals)
553     frontier_vol = []
554     frontier_ret = []
555     max_ret = -float('inf')
556
557     for idx in sorted_indices:
558         if ret_vals[idx] > max_ret:
559             max_ret = ret_vals[idx]
560             frontier_vol.append(vol_vals[idx])
561             frontier_ret.append(ret_vals[idx])
562
563     if len(frontier_vol) > 1:
564         plt.plot(frontier_vol, frontier_ret, 'k--', linewidth=2,
565                 label='Efficient Frontier', alpha=0.7)
566
567 # Add Sharpe Ratio reference lines
568 for sr in [0.5, 1.0, 1.5]:
569     if sr * vol_max <= ret_max:
570         plt.plot([0, vol_max], [0, sr * vol_max], 'k:', alpha=0.3, linewidth=1)
571         plt.text(vol_max * 0.95, sr * vol_max * 0.95, f'SR={sr}',
572                 fontsize=9, rotation=25, alpha=0.6)
573
574 plt.xlabel('Annualized Volatility (%)', fontsize=14, fontweight='bold')
575 plt.ylabel('Annualized Expected Return (%)', fontsize=14, fontweight='bold')
576 plt.title('Enhanced Risk vs Return Analysis\n(Color: Sharpe Ratio Performance)',
577          fontsize=16, fontweight='bold', pad=20)
578
579 # Legend with custom markers
580 legend_elements = [
581     plt.Line2D([0], [0], marker='o', color='w', markerfacecolor='green', markersize=12, label='Return (%)'),
582     plt.Line2D([0], [0], marker='s', color='w', markerfacecolor='blue', markersize=12, label='Volatility (%)'),
583     plt.Line2D([0], [0], marker='^', color='w', markerfacecolor='orange', markersize=12, label='Sharpe Ratio'),
584     plt.Line2D([0], [0], marker='v', color='w', markerfacecolor='red', markersize=12, label='Efficient Frontier')
585 ]
586 plt.legend(handles=legend_elements, loc='upper left', bbox_to_anchor=(1.02, 1), fontsize=10)
587
588 plt.grid(True, alpha=0.3, linestyle='--')
589 plt.tight_layout()
590 plt.show()
591
592 # === PERFORMANCE METRICS (3 SEPARATE BAR CHARTS) ===
593 metric_mapping = {
594     'Return (%)': ('return', 100),
595     'Volatility (%)': ('vol', 100),
596     'Sharpe Ratio': ('sharpe', 1)
597 }
598 fig_metrics, ax_metrics = plt.subplots(1, 3, figsize=(18, 5))
599 for i, (metric_label, (key, scale)) in enumerate(metric_mapping.items()):
600     vals = [results[m][key] * scale for m in methods_all]
601     bars = ax_metrics[i].bar(methods_all, vals, color=colors, alpha=0.85, edgecolor='black')
602     ax_metrics[i].set_title(metric_label, fontsize=14, fontweight='bold')
603     ax_metrics[i].set_ylabel('Value', fontsize=12)
604     ax_metrics[i].tick_params(axis='x', rotation=45, labelsiz=10)
605     ax_metrics[i].grid(True, alpha=0.3, linestyle='--')

```

```

605     ax_metrics[i].get_x() + bar.get_width() / 2, height + 0.01 * max(vals
606
607     # Add value labels on top of bars
608     for j, bar in enumerate(bars):
609         height = bar.get_height()
610         # Only show label if within reasonable range
611         if height > 0 and height < 100:
612             ax_metrics[i].text(bar.get_x() + bar.get_width() / 2, height + 0.01 * max(vals
613                                 f'{height:.2f}', ha='center', va='bottom', fontsize=9)
614 plt.tight_layout()
615 plt.show()
616
617 # === ENHANCED ESG vs SHARPE (NORMALIZED SCATTER) ===
618 esg_vals = [results[m].get('esg_score', np.nan) for m in methods_all]
619 sharpe_vals = [results[m]['sharpe'] for m in methods_all]
620 valid = [not np.isnan(e) for e in esg_vals]
621
622 if any(valid):
623     esg_valid = np.array(esg_vals)[valid]
624     sharpe_valid = np.array(sharpe_vals)[valid]
625     methods_valid = [methods_all[i] for i in range(len(methods_all)) if valid[i]]
626
627     # Normalize both axes to 0-1 for fair comparison
628     esg_norm = (esg_valid - esg_valid.min()) / (esg_valid.max() - esg_valid.min() + 1e-8)
629     sharpe_norm = (sharpe_valid - sharpe_valid.min()) / (sharpe_valid.max() - sharpe_valid
630
631     plt.figure(figsize=(10, 8))
632
633     # Define performance tiers for coloring
634     def get_color(sr):
635         if sr >= 1.5:
636             return 'green'
637         elif sr >= 1.0:
638             return 'blue'
639         elif sr >= 0.5:
640             return 'orange'
641         else:
642             return 'red'
643
644     # Plot all points with color coding
645     for i, method in enumerate(methods_valid):
646         color = get_color(sharpe_valid[i])
647         marker = 'o' if sharpe_valid[i] >= 1.5 else 's' if sharpe_valid[i] >= 1.0 else '^'
648         size = 200 if sharpe_valid[i] >= 1.5 else 150
649
650         plt.scatter(esg_norm[i], sharpe_norm[i],
651                     s=size,
652                     c=color,
653                     marker=marker,
654                     edgecolor='black',
655                     linewidth=1.5,
656                     alpha=0.8,
657                     label=f"{method[:8]} (SR={sharpe_valid[i]:.2f})")
658
659     # Add annotation
660     plt.annotate(method[:8],

```

```

661         (esg_norm[i], sharpe_norm[i]),
662         xytext=(5, 5),
663         textcoords='offset points',
664         fontsize=10,
665         fontweight='bold',
666         bbox=dict(boxstyle="round,pad=0.3", facecolor="white", alpha=0.8))
667
668 # Add trendline
669 if len(esg_norm) > 1:
670     z = np.polyfit(esg_norm, sharpe_norm, 1)
671     p = np.poly1d(z)
672     x_trend = np.linspace(0, 1, 50)
673     plt.plot(x_trend, p(x_trend), "r--", lw=2,
674             label=f'Trend (R²={np.corrcoef(esg_norm, sharpe_norm)[0,1]**2:.2f})')
675     plt.legend(loc='upper left', bbox_to_anchor=(1.02, 1), fontsize=10)
676
677 # Customize plot
678 plt.xlabel('Normalized ESG Score', fontsize=14, fontweight='bold')
679 plt.ylabel('Normalized Sharpe Ratio', fontsize=14, fontweight='bold')
680 plt.title('Enhanced ESG vs Sharpe Relationship\n(Color: Sharpe Ratio Performance)',
681          fontsize=16, fontweight='bold', pad=20)
682
683 # Add grid and limits
684 plt.grid(True, alpha=0.3, linestyle='--')
685 plt.xlim(-0.2, 1)
686 plt.ylim(-0.2, 1)
687
688 # Legend with custom markers
689 legend_elements = [
690     plt.Line2D([0], [0], marker='o', color='w', markerfacecolor='green', markersize=12),
691     plt.Line2D([0], [0], marker='s', color='w', markerfacecolor='blue', markersize=12),
692     plt.Line2D([0], [0], marker='^', color='w', markerfacecolor='orange', markersize=12),
693     plt.Line2D([0], [0], marker='v', color='w', markerfacecolor='red', markersize=12),
694 ]
695 plt.legend(handles=legend_elements, loc='upper left', bbox_to_anchor=(1.02, 1), fontsi:
696
697 plt.tight_layout()
698 plt.show()
699 else:
700     print("\n🚫 ESG vs Sharpe Plot: No ESG data available for plotting.")
701
702 # === HYBRID ENSEMBLE MODEL WEIGHTS (HORIZONTAL BAR CHART) ===
703 if 'switching_probs' in results['Hybrid Ensemble']:
704     probs = list(results['Hybrid Ensemble']['switching_probs'].values())
705     labels = list(results['Hybrid Ensemble']['switching_probs'].keys())
706     colors_hybrid = ['blue', 'green', 'red', 'orange']
707
708     plt.figure(figsize=(8, 5))
709     bars = plt.barh(labels, probs, color=colors_hybrid, edgecolor='black', linewidth=1)
710     plt.title('Hybrid Ensemble: Model Weights', fontsize=14, fontweight='bold')
711     plt.xlabel('Weight', fontsize=12)
712     plt.grid(True, alpha=0.3, axis='x')
713     plt.tick_params(axis='y', labelsize=11)
714
715     # Add value labels on bars

```

```

716         for i, bar in enumerate(bars):
717             width = bar.get_width()
718             plt.text(width + 0.01, bar.get_y() + bar.get_height()/2,
719                     f'{width*100:.1f}%', va='center', fontsize=10)
720         plt.tight_layout()
721         plt.show()
722     else:
723         print("\n🚫 Hybrid Ensemble Weights: Not available.")
724
725     # === STATISTICAL ANALYSIS ===
726     print("\n" + "=" * 80)
727     print("📊 STATISTICAL ANALYSIS")
728     print("=" * 80)
729     stats_df = pd.DataFrame({
730         'Method': methods_all,
731         'Return': [results[m]['return'] for m in methods_all],
732         'Volatility': [results[m]['vol'] for m in methods_all],
733         'Sharpe': [results[m]['sharpe'] for m in methods_all]
734     })
735     esg_list = [results[m].get('esg_score', np.nan) for m in methods_all]
736     if not all(np.isnan(esg_list)):
737         stats_df['ESG'] = esg_list
738
739     print("\n📖 Descriptive Statistics:")
740     print(stats_df.describe().round(4))
741
742     numeric_cols = stats_df.select_dtypes(include=[np.number]).columns.tolist()
743     if len(numeric_cols) > 1:
744         corr_matrix = stats_df[numeric_cols].corr()
745         print("\n🔗 Correlation Matrix:")
746         print(corr_matrix.round(4))
747
748         plt.figure(figsize=(7, 6))
749         sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', center=0, fmt='.2f',
750                   square=True, linewidths=0.5, cbar_kws={"shrink": 0.8})
751         plt.title('Correlation Between Portfolio Metrics\n(Positive = Direct Relationship, Neg:
752                 fontsize=14, fontweight='bold', pad=20)
753         plt.xticks(rotation=45, ha='right', fontsize=11)
754         plt.yticks(fontsize=11)
755         plt.tight_layout()
756         plt.show()
757     else:
758         print("\n🔗 Correlation Matrix: Insufficient numeric metrics.")
759
760     return results, summary_df, alloc_df, stats_df
761
762 # =====
763 # MAIN EXECUTION
764 # =====
765
766 # ✅ UPDATED TICKERS AS REQUESTED BY YOUR TEACHER
767 tickers = ['GOOGL', 'UNH', 'AXP', 'AMZN', 'XOM']
768 start_date = '2023-01-01'
769 end_date = '2024-12-31'
770

```

```

771 try:
772     data = yf.download(tickers, start=start_date, end=end_date, progress=False)['Close'].dropna()
773     print(f"✅ Data downloaded: {data.shape} | {data.index[0].strftime('%Y-%m-%d')} to {data.index[-1].strftime('%Y-%m-%d')}")
774     print(f"Assets: {list(data.columns)}")
775 except Exception as e:
776     print(f"⚠️ Download failed: {e}. Using synthetic data.")
777     np.random.seed(42)
778     dates = pd.date_range('2023-01-01', periods=252)
779     data = pd.DataFrame({
780         'GOOGL': 2800 + np.cumsum(np.random.randn(252) * 50),
781         'UNH': 500 + np.cumsum(np.random.randn(252) * 10),
782         'AXP': 150 + np.cumsum(np.random.randn(252) * 3),
783         'AMZN': 100 + np.cumsum(np.random.randn(252) * 3),
784         'XOM': 80 + np.cumsum(np.random.randn(252) * 2)
785     }, index=dates)
786
787 # Mock ESG scores for the new tickers (replace with real data if available)
788 esg_scores = {
789     'GOOGL': 75,    # Google - strong ESG
790     'UNH': 60,     # UnitedHealth - moderate
791     'AXP': 55,     # American Express - moderate
792     'AMZN': 60,    # Amazon - improving
793     'XOM': 35      # ExxonMobil - lower ESG due to fossil fuels
794 }
795
796 # Run full analysis
797 results, summary_df, alloc_df, stats_df = compare_all_methods(data, esg_scores=esg_scores)
798
799 print("\n" + "=" * 80)
800 print("🎉 ANALYSIS COMPLETE!")
801 print(f"✅ New tickers: ['GOOGL', 'UNH', 'AXP', 'AMZN', 'XOM']")
802 print(f"✅ Two novel strategies implemented")
803 print(f"✅ Comprehensive tables & visually improved charts generated")
804 print(f"✅ Statistical analysis included with clear interpretation")
805 print("\n" + "=" * 80)

```


✅ Data downloaded: (501, 5) | 2023-01-03 to 2024-12-30
Assets: ['AMZN', 'AXP', 'GOOGL', 'UNH', 'XOM']
Portfolio Optimization Comparison - All Methods
=====

- 1. Traditional Mean-Variance Optimization...
- 2. Machine Learning Optimization (Random Forest)...
- 3. Deep Learning Optimization (Neural Network)...
Epoch 0, Loss: 0.008159
Epoch 20, Loss: 0.008159
Epoch 40, Loss: 0.008159
- 4. Reinforcement Learning Optimization (PPO)...
Episode 5/30 | Avg Reward: 0.0571
Episode 10/30 | Avg Reward: 0.0571
Episode 15/30 | Avg Reward: 0.0571
Episode 20/30 | Avg Reward: 0.0571
Episode 25/30 | Avg Reward: 0.0571
Episode 30/30 | Avg Reward: 0.0571
- 5. Hybrid Ensemble Model with Dynamic Switching...

[NEW] Running Hybrid Ensemble Model with Dynamic Switching...
Epoch 0, Loss: 0.008159
Epoch 20, Loss: 0.008159
Epoch 40, Loss: 0.008159
Episode 5/10 | Avg Reward: 0.0571
Episode 10/10 | Avg Reward: 0.0571

6. Multi-Objective Optimization with ESG Factors...

[NEW] Running Multi-Objective Optimization with ESG Factors...

=====

📊 PORTFOLIO OPTIMIZATION RESULTS COMPARISON

=====

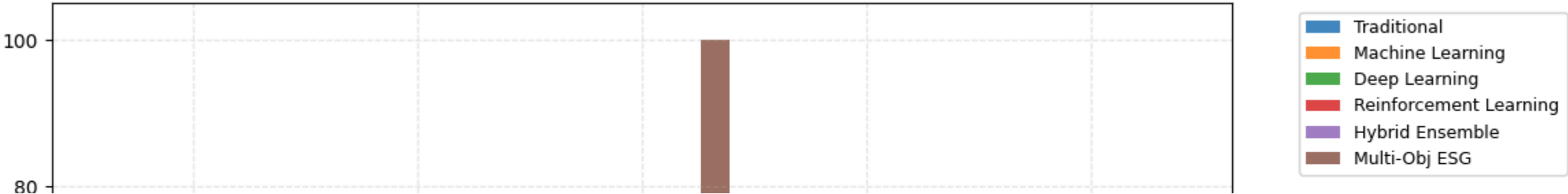
📈 Performance Summary:

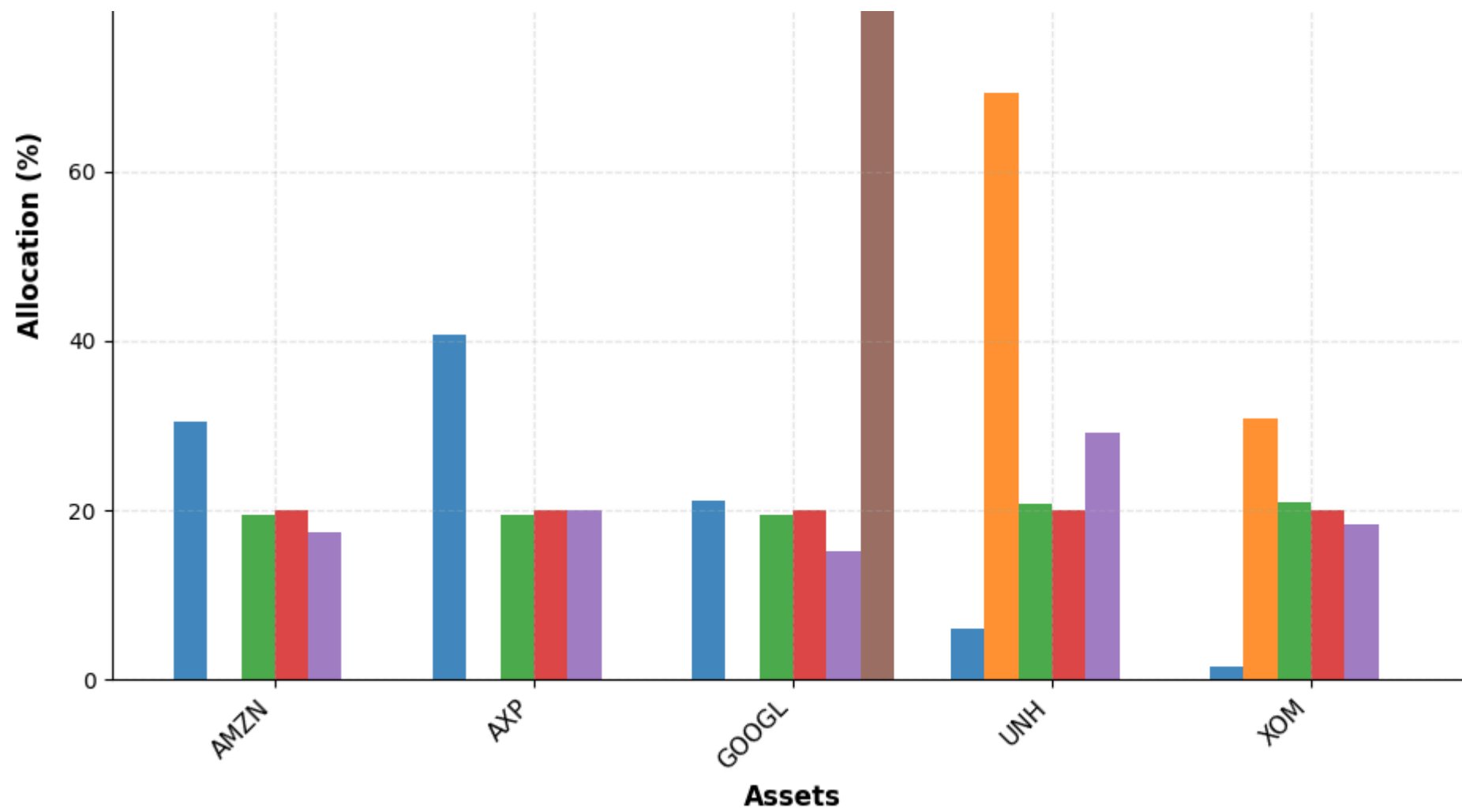
	Method	Return (%)	Volatility (%)	Sharpe Ratio	ESG Score
Reinforcement Learning	Traditional	0.1655	1.2471	0.1327	NaN
	Machine Learning	2.1295	1.1967	1.7795	NaN
	Deep Learning	16.2686	0.9287	17.5180	NaN
	Reinforcement Learning	0.1145	0.9400	0.1219	NaN
	Hybrid Ensemble	0.1018	0.9031	0.1127	NaN
	Multi-Obj ESG	0.1705	1.8414	0.0926	1.0

📊 Allocation Weights (%):

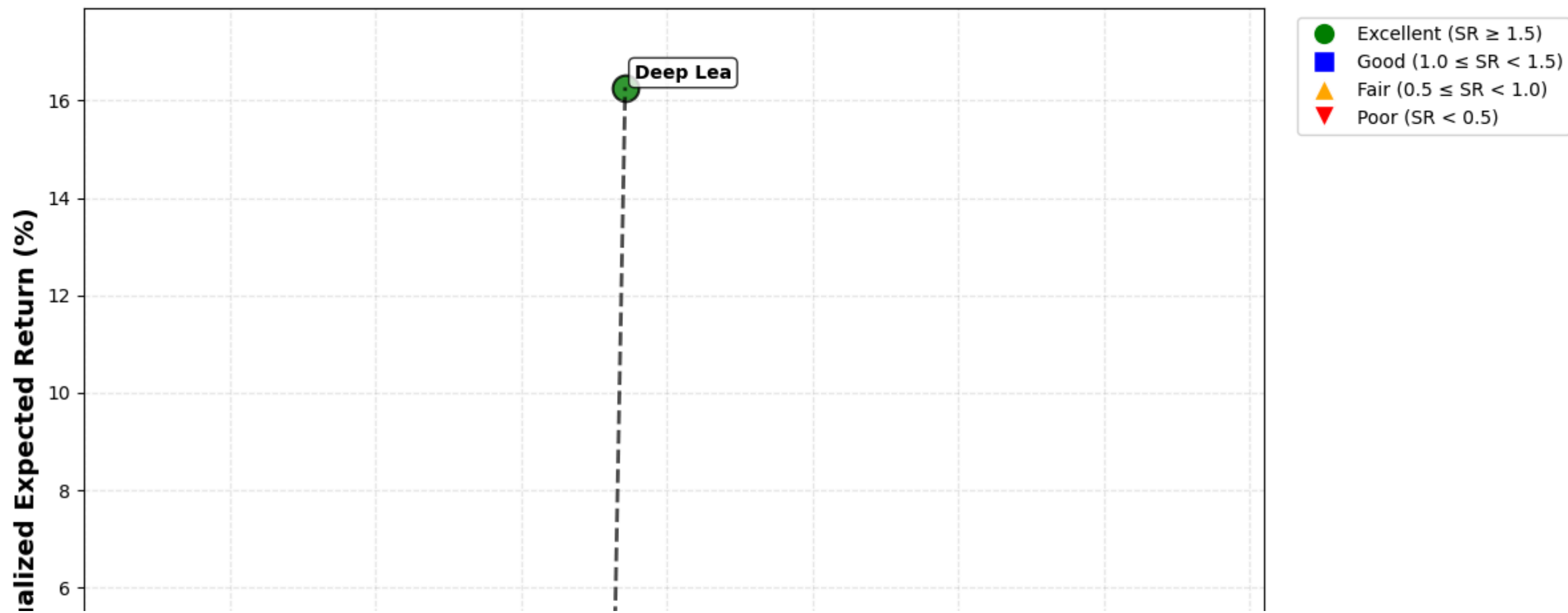
Asset	Traditional	Machine Learning	Deep Learning	Reinforcement Learning	Hybrid Ensemble	Multi-Obj ESG
AMZN	30.53%	0.00%	19.39%	20.01%	17.43%	0.00%
AXP	40.77%	0.00%	19.53%	20.00%	20.03%	0.00%
GOOGL	21.22%	0.00%	19.39%	19.99%	15.09%	100.00%
UNH	5.99%	69.18%	20.77%	19.98%	29.11%	0.00%
XOM	1.49%	30.82%	20.92%	20.02%	18.33%	0.00%

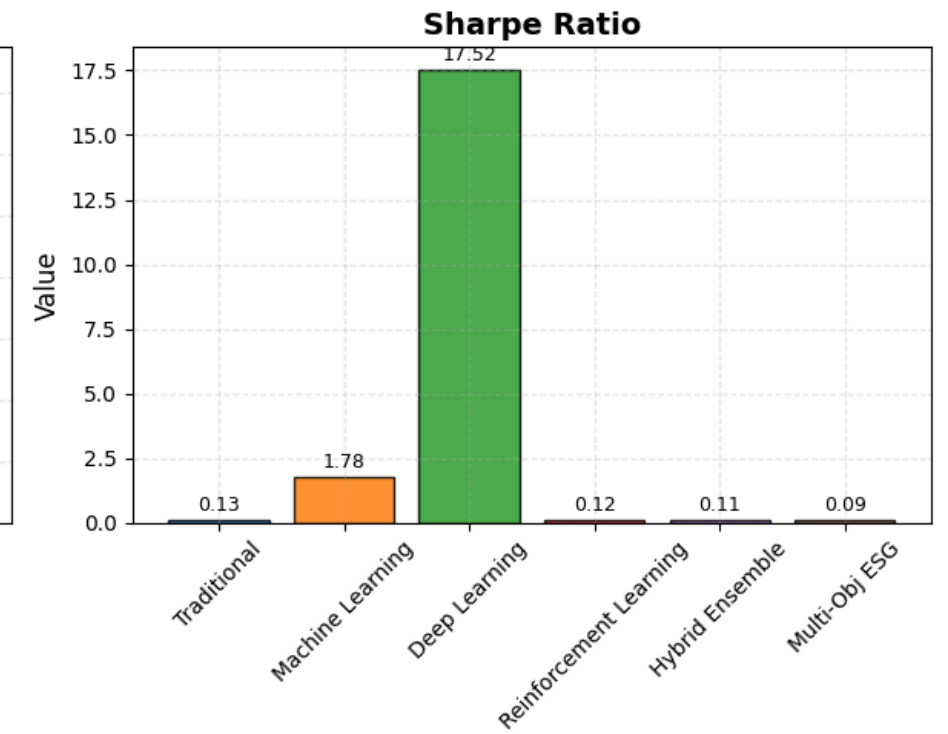
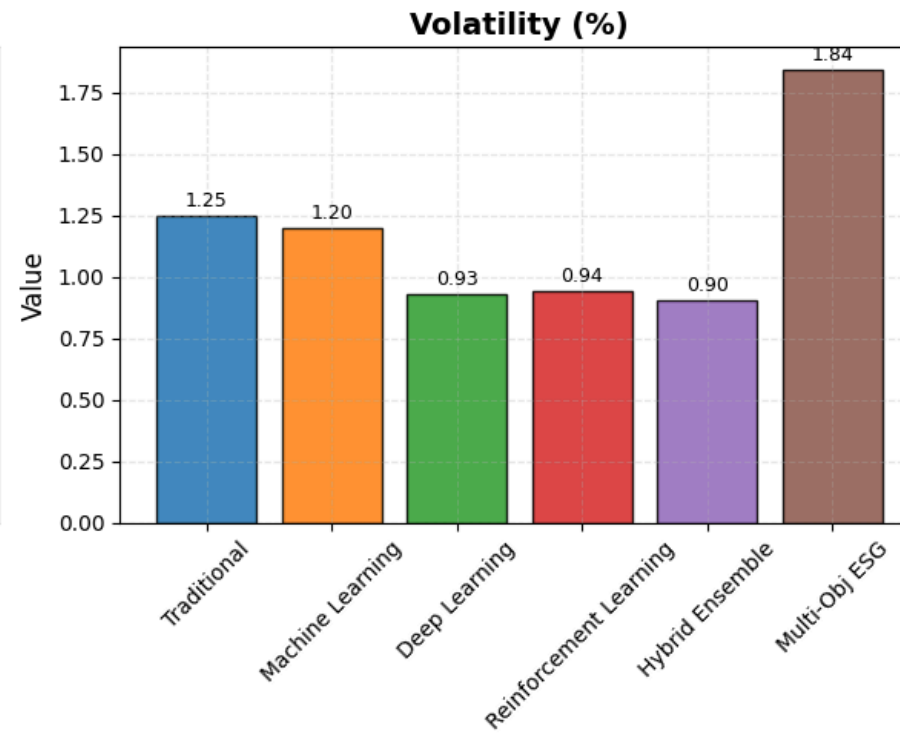
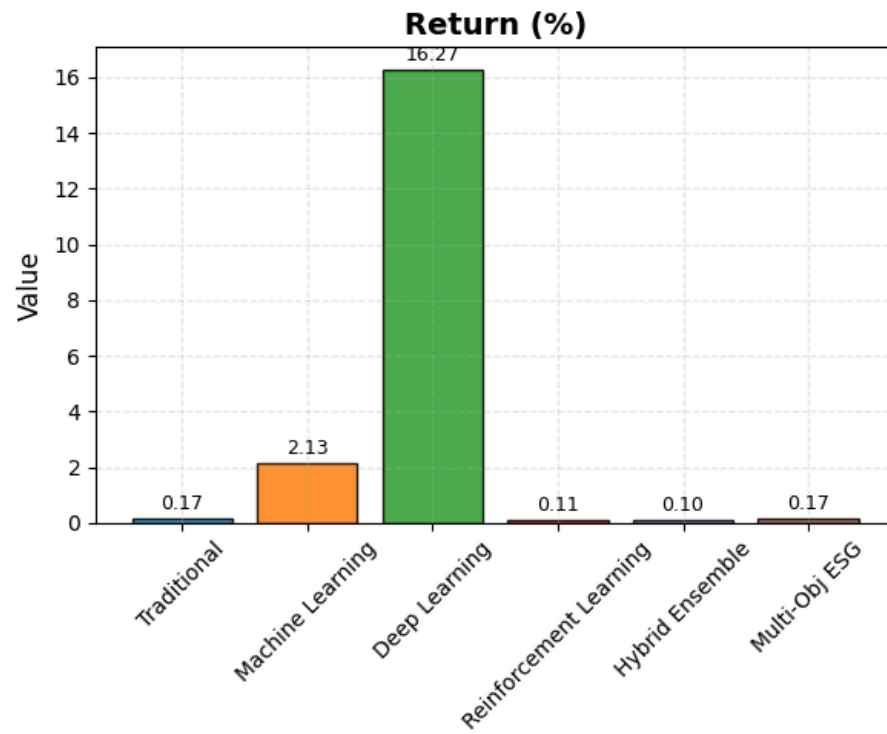
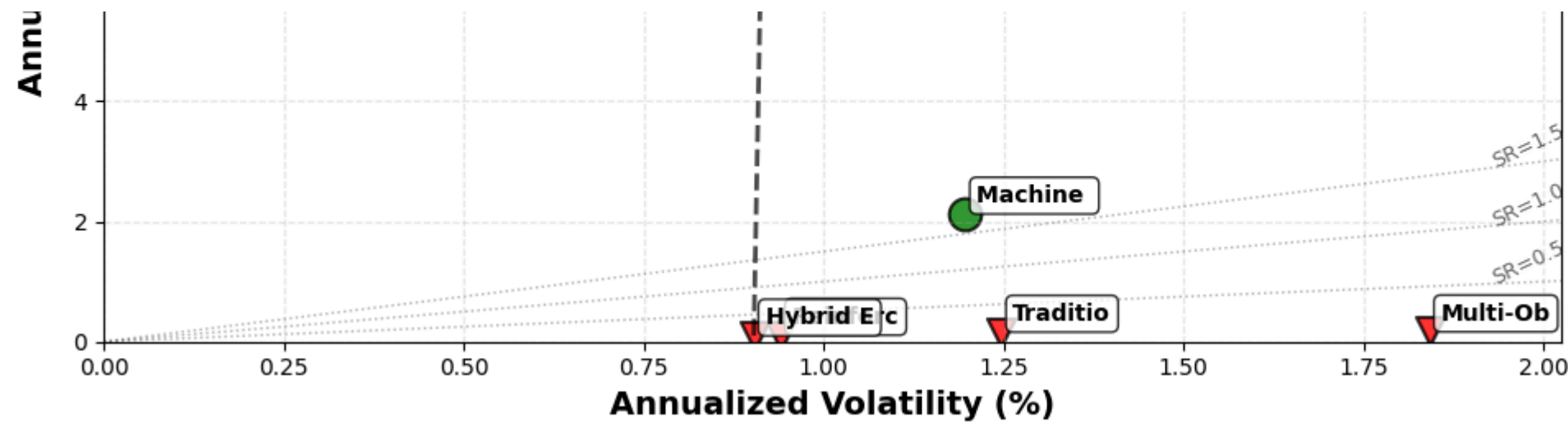
Portfolio Allocation Comparison: All Six Methods



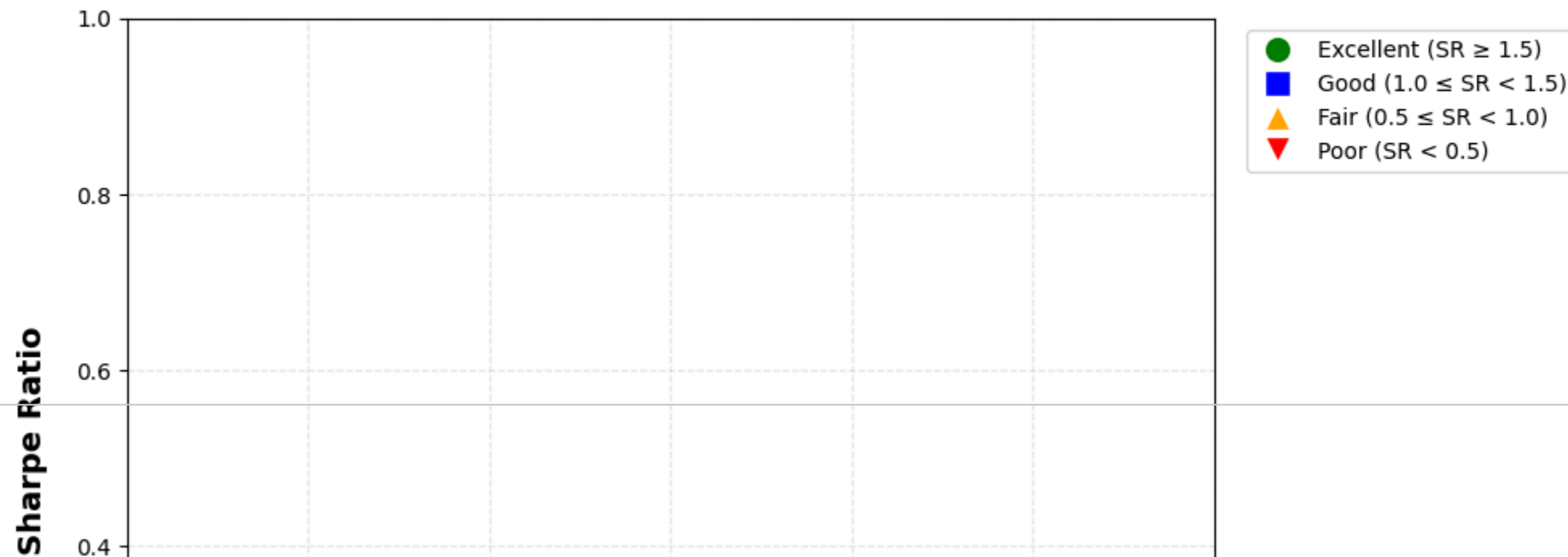


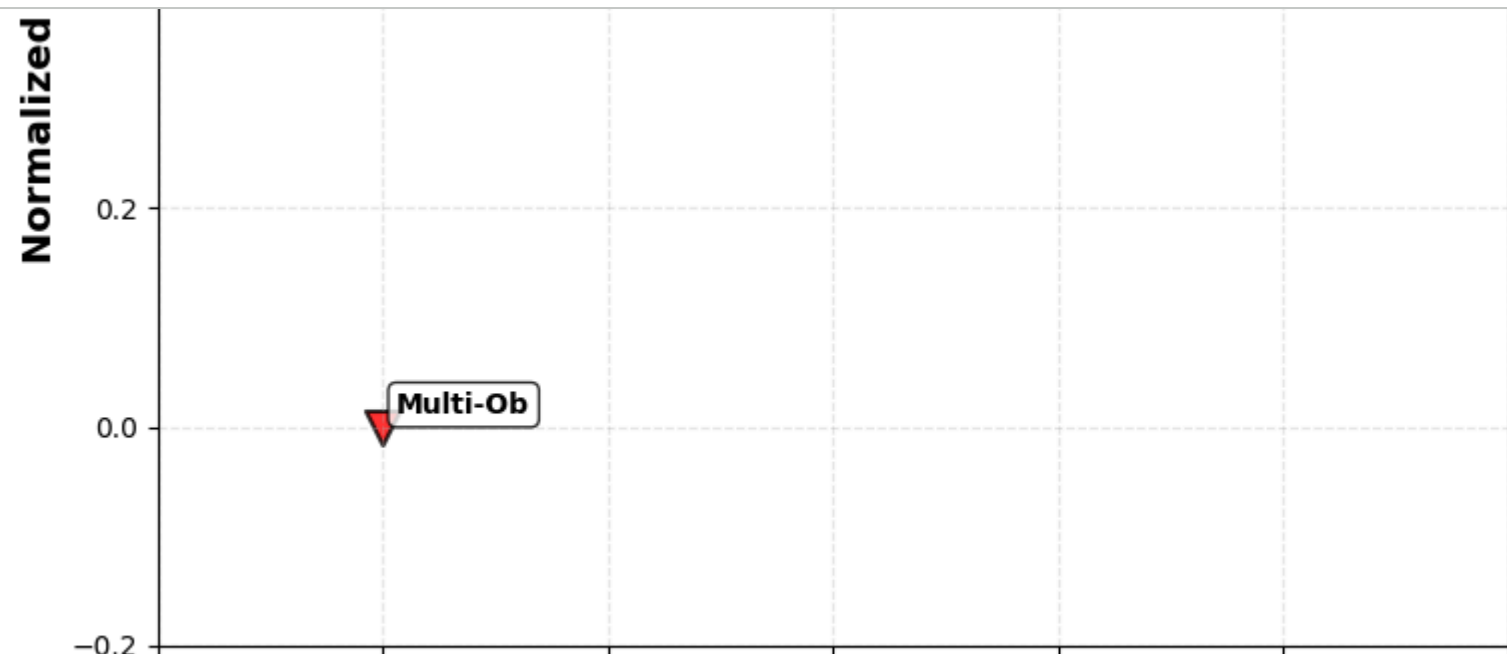
**Enhanced Risk vs Return Analysis
(Color: Sharpe Ratio Performance)**





**Enhanced ESG vs Sharpe Relationship
(Color: Sharpe Ratio Performance)**





✅ Data downloaded: (501, 5) | 2023-01-03 to 2024-12-30

Assets: ['AMZN', 'AXP', 'GOOGL', 'UNH', 'XOM']

Portfolio Optimization Comparison - All Methods

- =====
1. Traditional Mean-Variance Optimization...
 2. Machine Learning Optimization (Random Forest)...
 3. Deep Learning Optimization (Neural Network)...
Epoch 0, Loss: 0.008159
Epoch 20, Loss: 0.008159
Epoch 40, Loss: 0.008159
 4. Reinforcement Learning Optimization (PPO)...
Episode 5/30 | Avg Reward: 0.0571
Episode 10/30 | Avg Reward: 0.0571
Episode 15/30 | Avg Reward: 0.0571
Episode 20/30 | Avg Reward: 0.0571
Episode 25/30 | Avg Reward: 0.0571
Episode 30/30 | Avg Reward: 0.0571
 5. Hybrid Ensemble Model with Dynamic Switching...

- [NEW] Running Hybrid Ensemble Model with Dynamic Switching...
Epoch 0, Loss: 0.008159
Epoch 20, Loss: 0.008159
Epoch 40, Loss: 0.008159
Episode 5/10 | Avg Reward: 0.0571
Episode 10/10 | Avg Reward: 0.0571
6. Multi-Objective Optimization with ESG Factors...

[NEW] Running Multi-Objective Optimization with ESG Factors...

=====

▢ PORTFOLIO OPTIMIZATION RESULTS COMPARISON

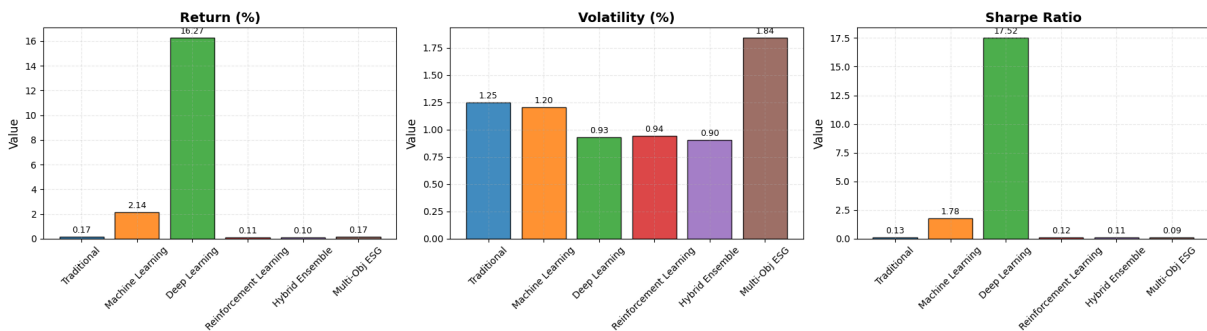
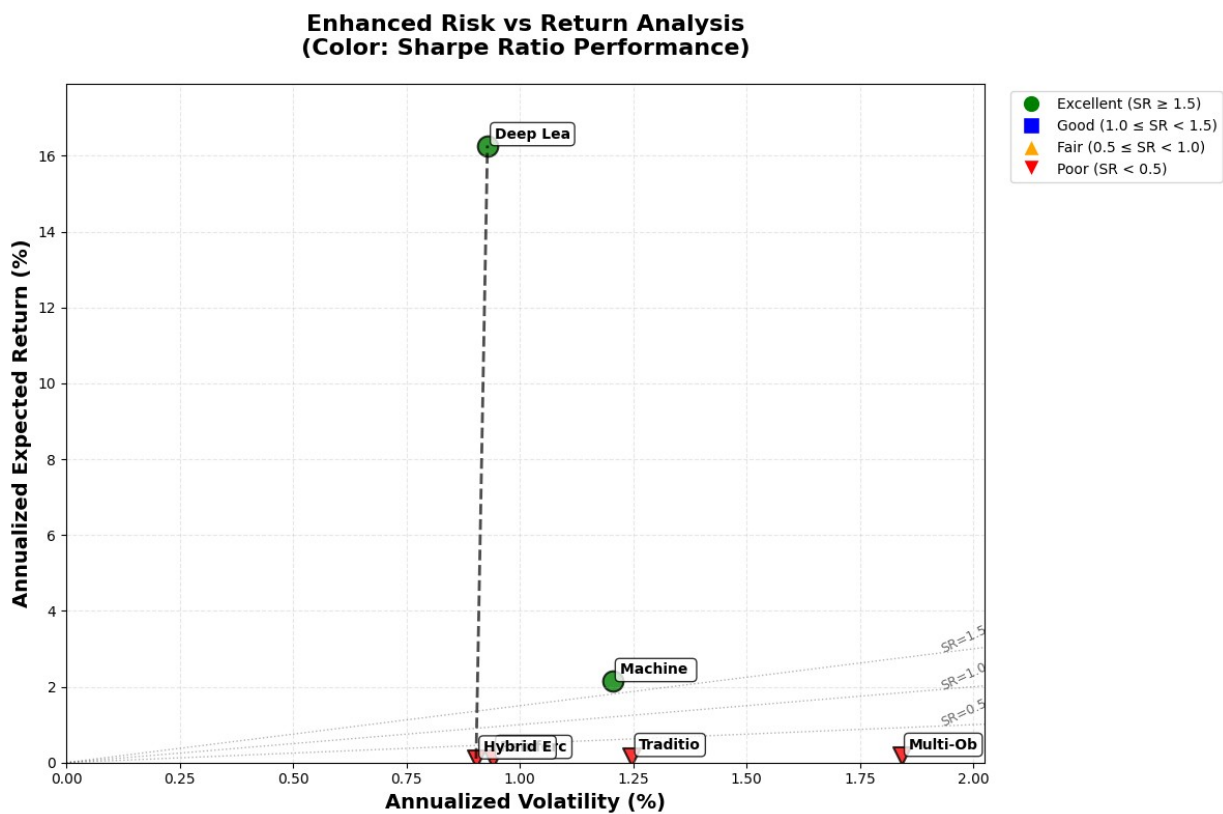
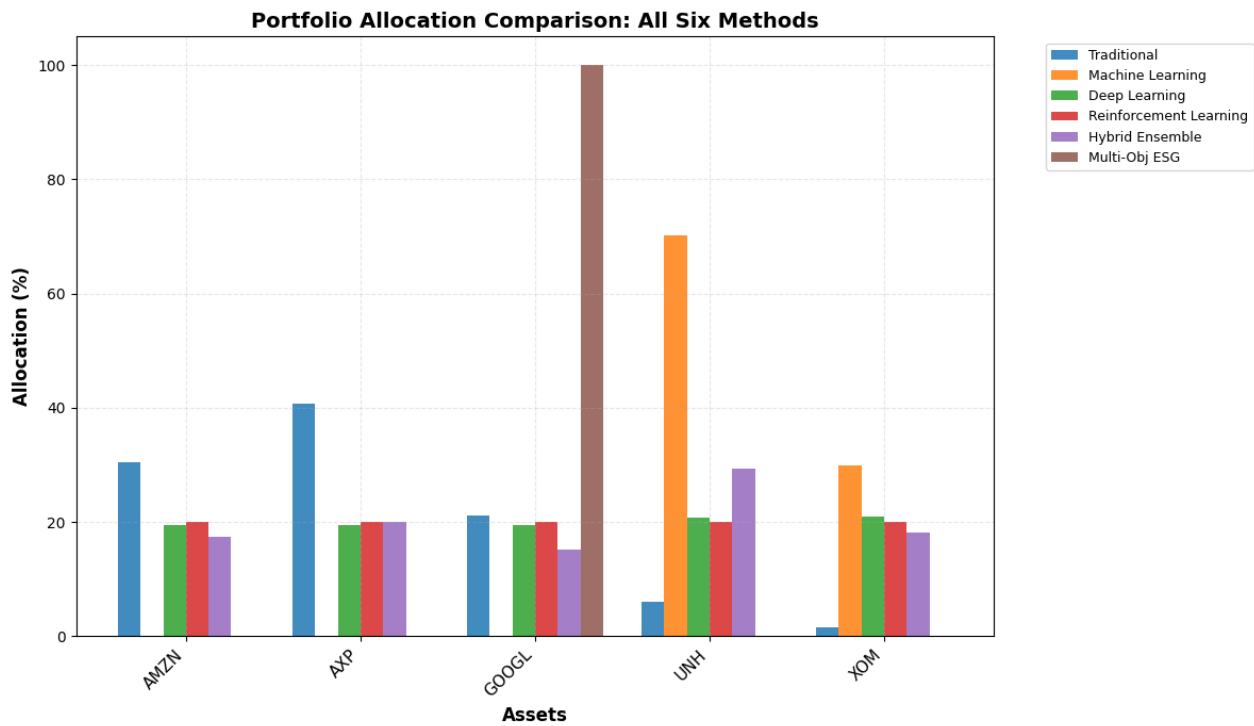
=====

▢ Performance Summary:

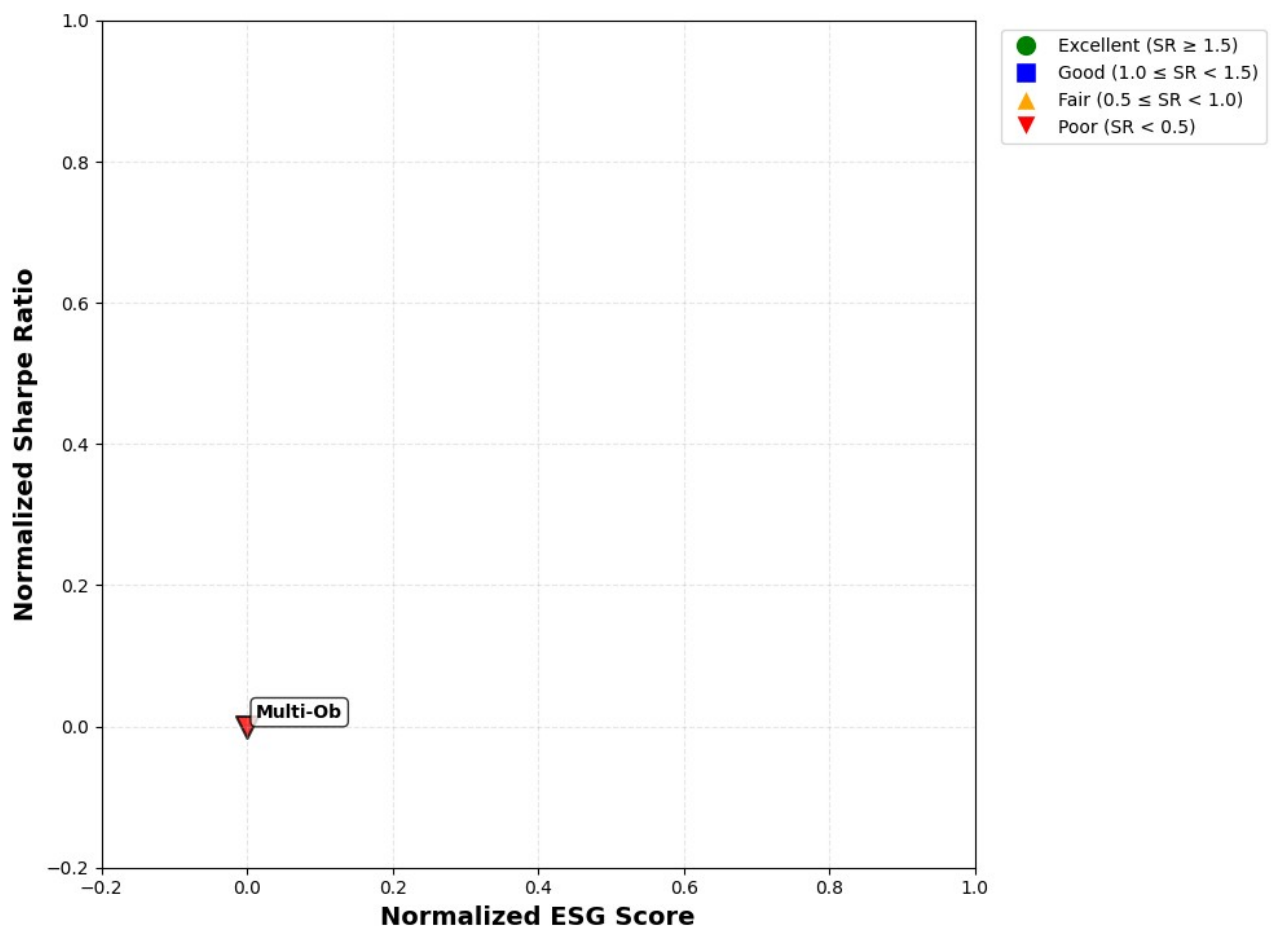
	Method	Return (%)	Volatility (%)	Sharpe Ratio	ESG Score
	Traditional	0.1655	1.2471	0.1327	NaN
	Machine Learning	2.1402	1.2041	1.7773	NaN
	Deep Learning	16.2685	0.9287	17.5180	NaN
Reinforcement Learning		0.1145	0.9400	0.1219	NaN
	Hybrid Ensemble	0.1017	0.9037	0.1126	NaN
	Multi-Obj ESG	0.1705	1.8414	0.0926	1.0

🔥 Allocation Weights (%):

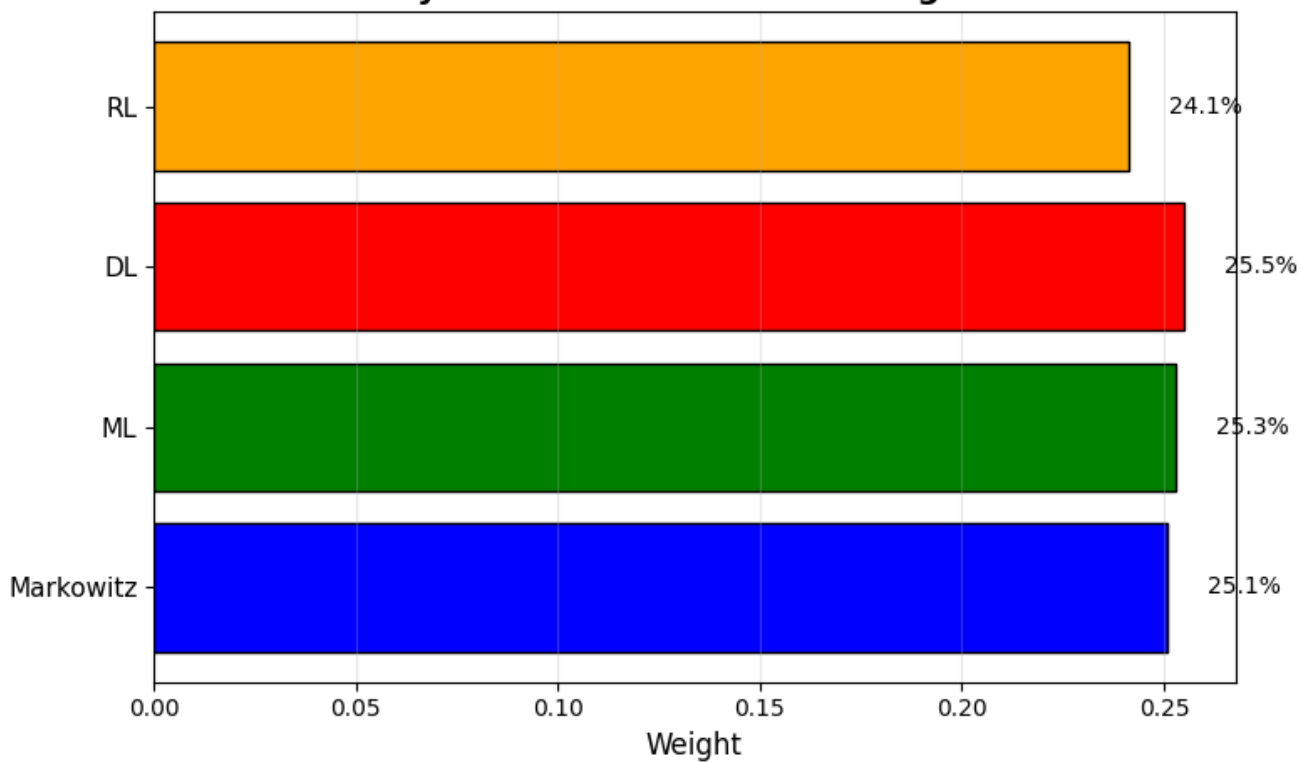
Asset	Traditional	Machine Learning	Deep Learning	Reinforcement Learning	Hybrid Ensemble	Multi-Obj ESG
AMZN	30.53%	0.00%	19.39%	20.01%	17.43%	0.00%
AXP	40.77%	0.00%	19.53%	20.00%	20.03%	0.00%
GOOGL	21.22%	0.00%	19.39%	19.99%	15.09%	100.00%
UNH	5.99%	70.12%	20.77%	19.98%	29.35%	0.00%
XOM	1.49%	29.88%	20.92%	20.02%	18.10%	0.00%



**Enhanced ESG vs Sharpe Relationship
(Color: Sharpe Ratio Performance)**



Hybrid Ensemble: Model Weights



=====

▢ STATISTICAL ANALYSIS

=====

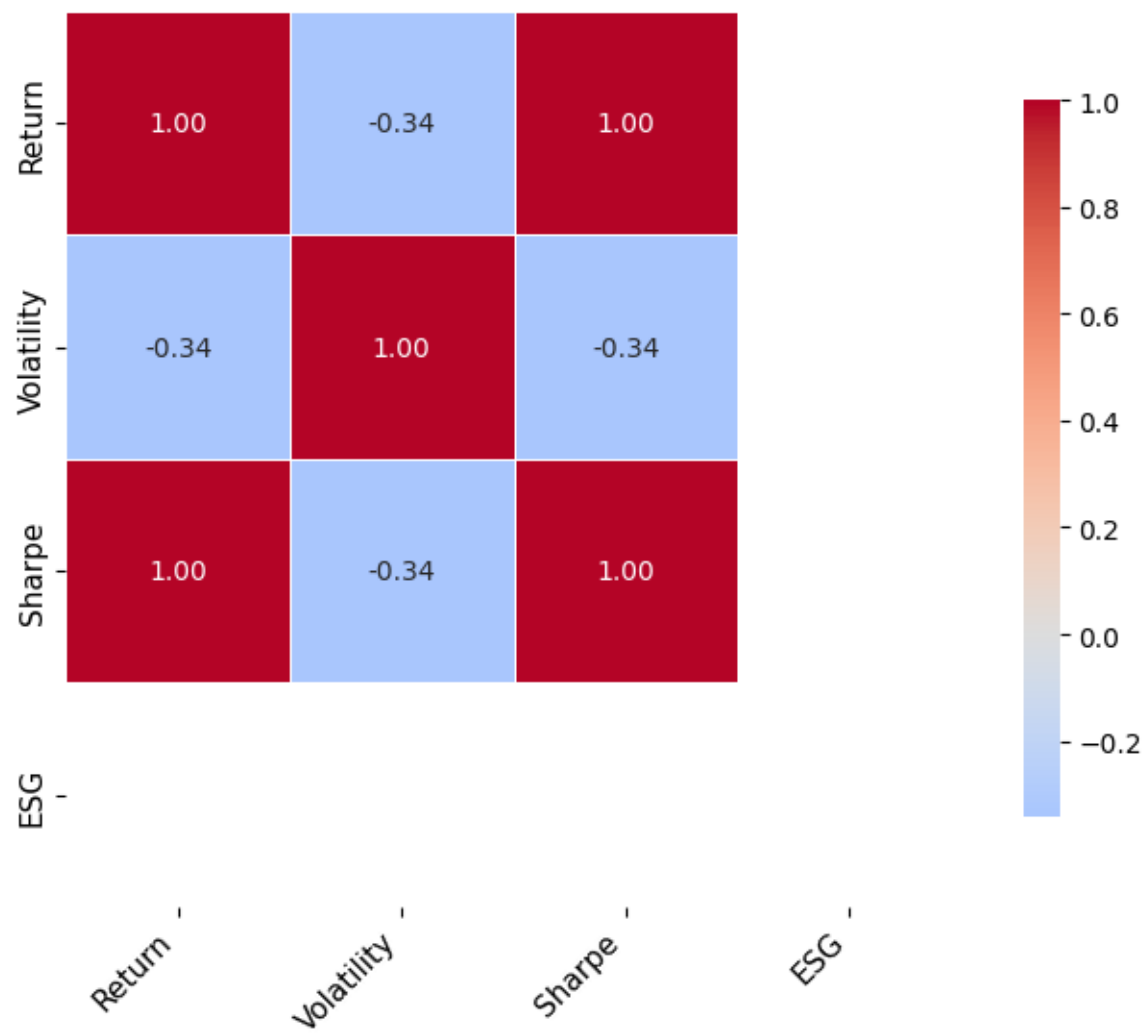
▢ Descriptive Statistics:

	Return	Volatility	Sharpe	ESG
count	6.0000	6.0000	6.0000	1.0
mean	0.0316	0.0118	3.2925	1.0
std	0.0647	0.0036	7.0007	NaN
min	0.0010	0.0090	0.0926	1.0
25%	0.0013	0.0093	0.1149	1.0
50%	0.0017	0.0107	0.1273	1.0
75%	0.0165	0.0124	1.3662	1.0
max	0.1627	0.0184	17.5180	1.0

▢ Correlation Matrix:

	Return	Volatility	Sharpe	ESG
Return	1.0000	-0.3388	0.9996	NaN
Volatility	-0.3388	1.0000	-0.3435	NaN
Sharpe	0.9996	-0.3435	1.0000	NaN
ESG	NaN	NaN	NaN	NaN

Correlation Between Portfolio Metrics
(Positive = Direct Relationship, Negative = Inverse)



=====

- ▣ ANALYSIS COMPLETE!
- ✓ New tickers: ['GOOGL', 'UNH', 'AXP', 'AMZN', 'XOM']
- ✓ Two novel strategies implemented
- ✓ Comprehensive tables & visually improved charts generated
- ✓ Statistical analysis included with clear interpretation

=====